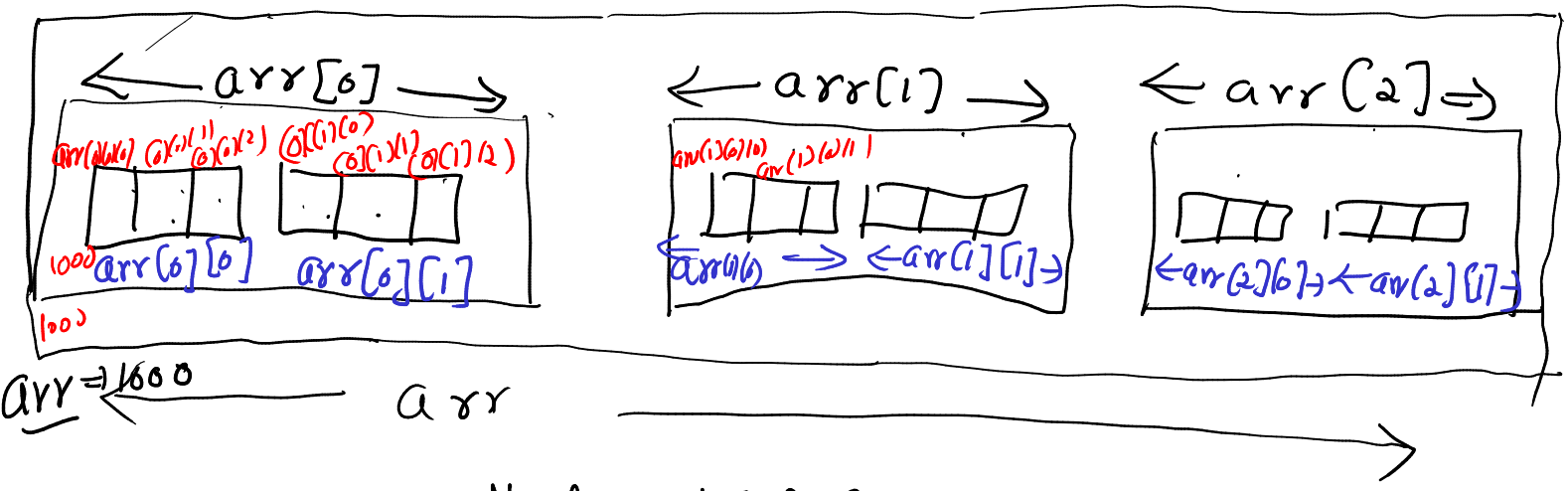
3D Array

datatype arrayname [x] [y] [z];

- x → no of 2D arrays
- y → no of 1D arrays in each 2D array
- z → no of elements in each 1D array.

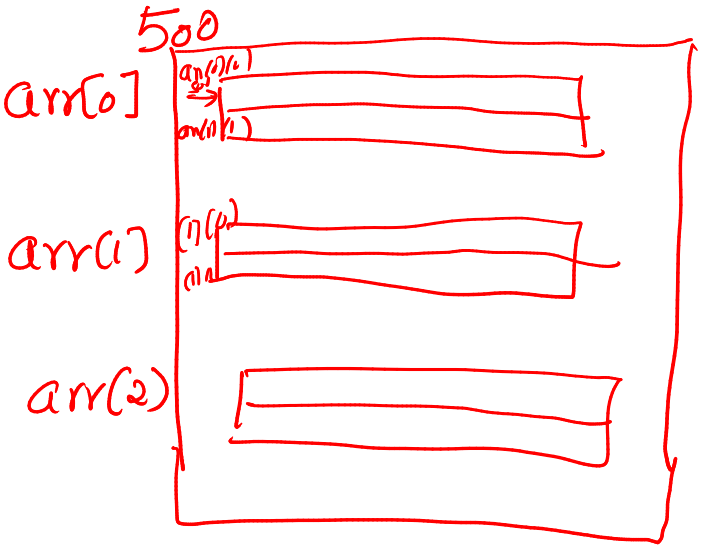
int arr [3] [2] [3];



3D Array ⇒ Collection of 2D array.

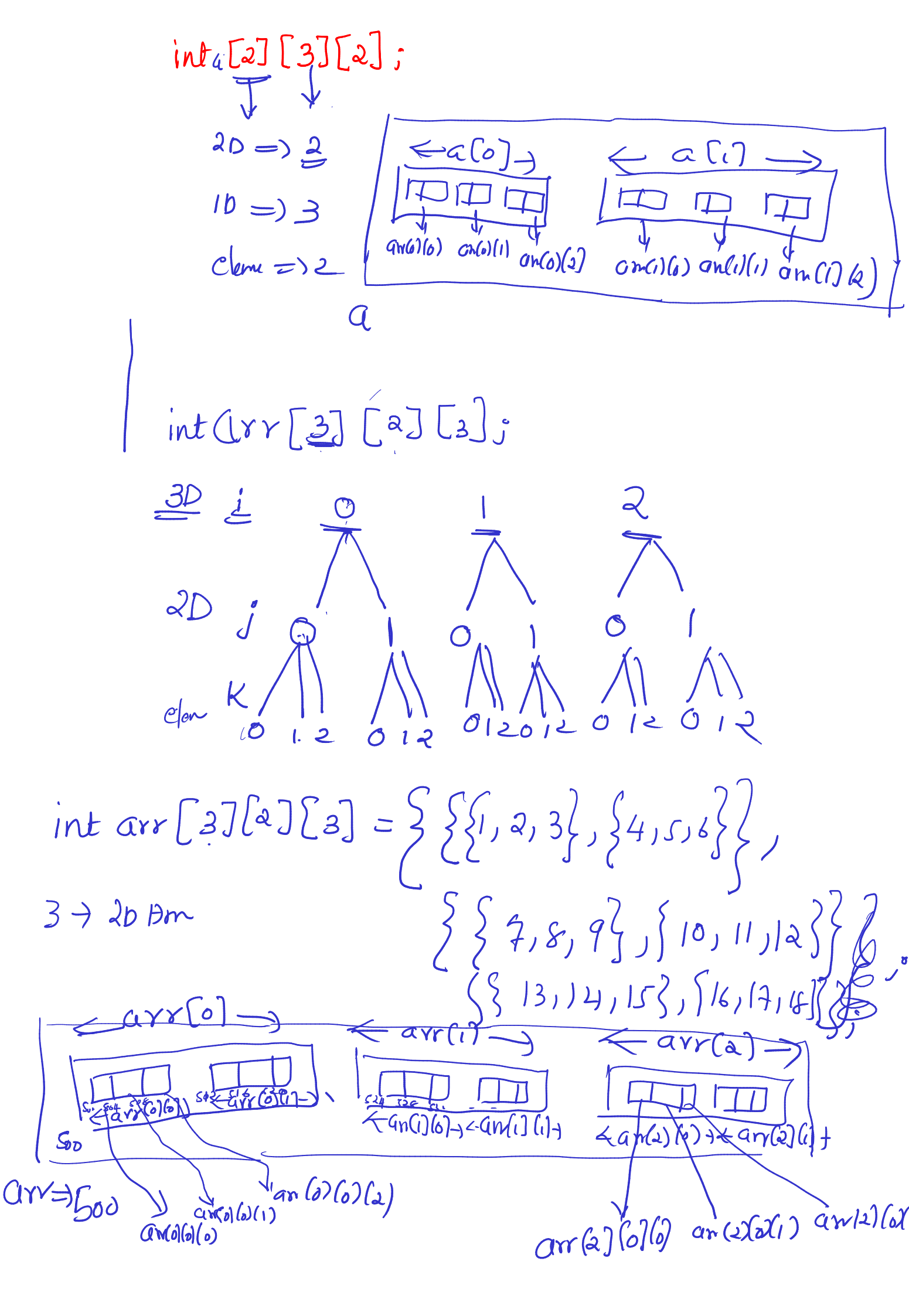
2D array ⇒ Collection of 1D arrays

1D arrays ⇒ elements.

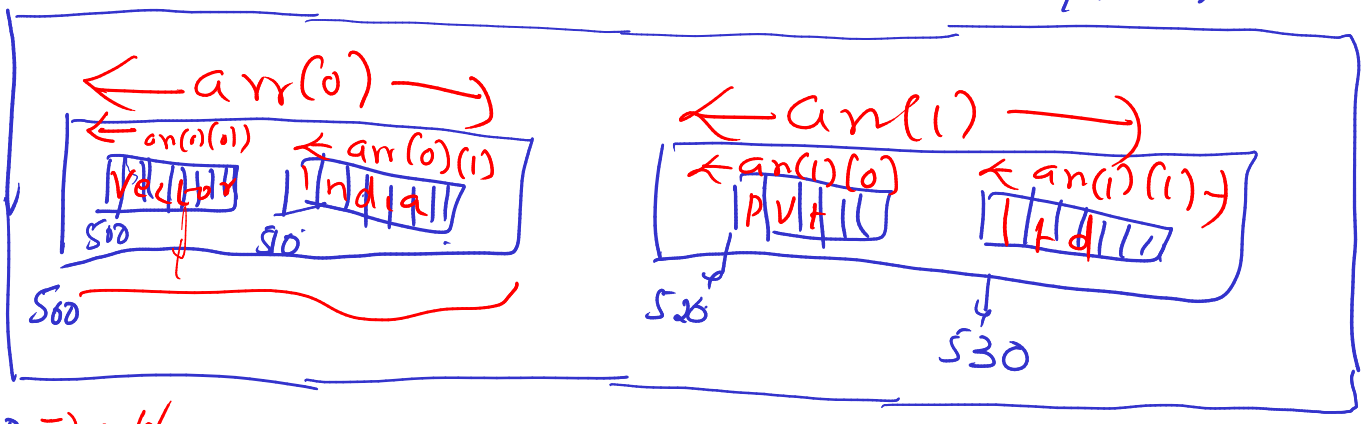


At 1000

- ↳ 3D array
- ↳ 2D array
- ↳ 1D array
- ↳ 0th element


int a[2][3][2];
2D => 2
1D => 3
elem => 2
a[0]
a[1]
a[0][0] a[0][1] a[0][2]
a[1][0] a[1][1] a[1][2]
a
int arr[3][2][3];
3D i
2D j
elem K
0 1 2
0 1
int arr[3][2][3] = { {{1, 2, 3}, {4, 5, 6}},
{{7, 8, 9}, {10, 11, 12}}
{{13, 14, 15}, {16, 17, 18}} };
3 → 2D Arr
arr[0]
arr[1]
arr[2]
arr[0][0]
arr[0][1]
arr[1][0]
arr[1][1]
arr[2][0]
arr[2][1]
500
arr => 500
arr[0][0][0]
arr[0][0][1]
arr[0][0][2]
arr[2][0][0]
arr[2][0][1]
arr[2][0][2]

char arr[2][2][10] = {{"vector", "india"}, {"pvt", "ltd"}};



500 => arr

```
for(i=0; i<2; i++)
{
    for(j=0; j<2; j++)
    {
        printf("%s", arr[i][j]);
    }
}
```

0 j<0

j<2

arr(0)(0) => vector

j=1

arr[0][1] = india

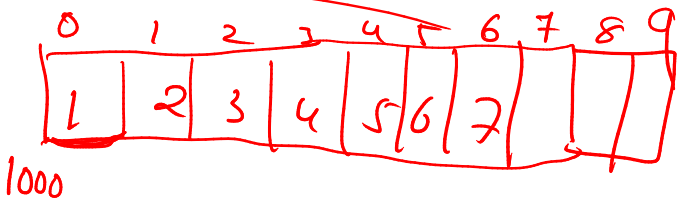
i=1

arr[1][0] = pvt

arr(1)(1) = ltd.

---

int arr[10];

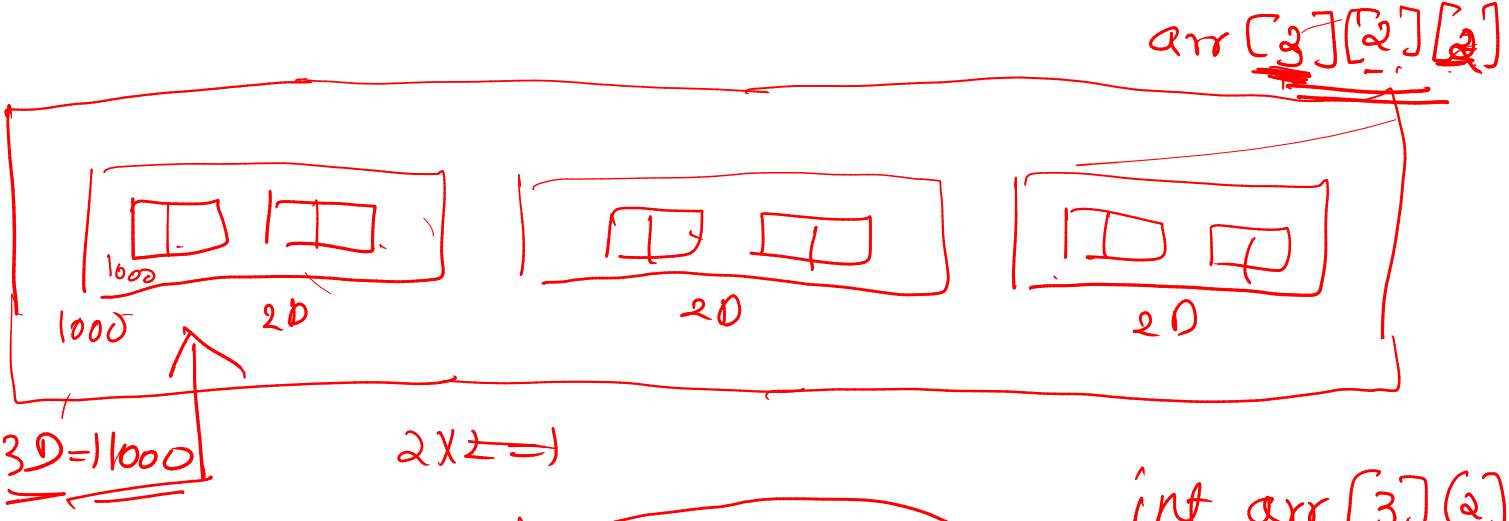


array passing => address passing

pointer can hold address.

int (*p);

pointer to an integer



int arr[3][2][2]

int (*p)[2][2]

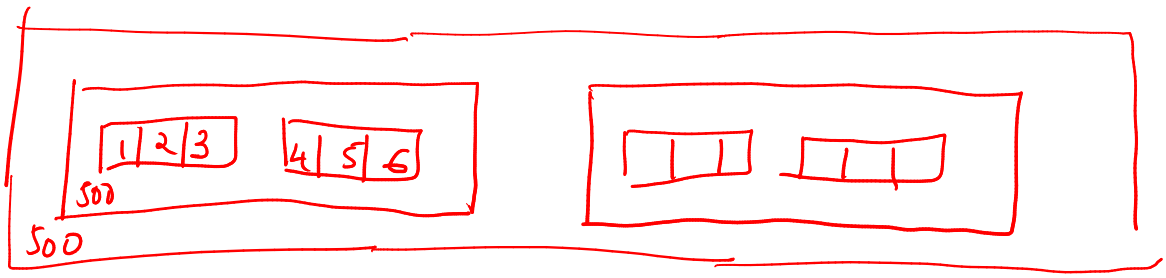
int arr[2][3][4]

int (*p)[3][4]

*p ⟹ p => 8 byte
*p => 16

(*p)[2][2]

int arr[2][2][3]



arr=500

display(arr);

int (*p)[2][3];

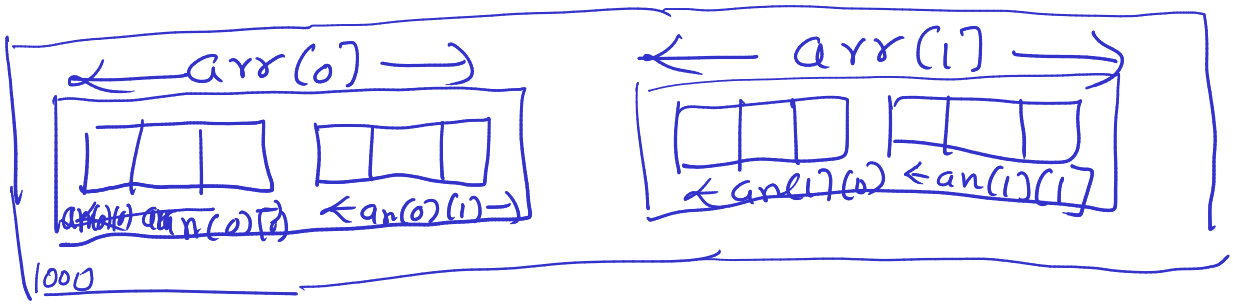
sizeof(p) => 8 by

sizeof(*p) = 24

sizeof(**p) = 12

sizeof(***p)=4

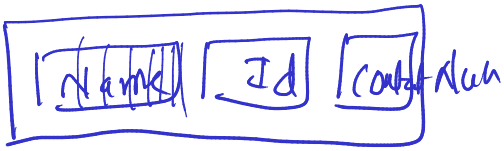
int ~~char~~ arr[2][2][3]



arr[] = 1000

Contact Database

Student ⇒ Name
emailId
contactNumber



10 ⇒ Students

char arr[10][3][20];

cnt = 0

